AIE BOOTCAMP / COMPANION GUIDE

Engineering Standards

How to build the AI parts like an engineer

Async • DI • Singletons • Caching • Config • Types • Errors • Hygiene • Tests

SE Factory / Hasan

How to Read This

This is a companion to the Week 4 project brief. The brief tells you what to build. This document tells you how to build it like an engineer — and, more importantly, why each of these standards exists at all.

Every chapter follows the same shape: the problem the standard solves, the antipattern you will write if no one stops you, the right pattern with runnable code, and a short note on when this matters in practice. Read it once cover to cover before you start coding. Reread the relevant chapter when you hit something you do not know how to do.

> *Frameworks change. Libraries get rewritten. The patterns in this document have outlived three generations of Python web frameworks already. Learn them once, carry them forever.*

None of this is theory. Every antipattern in here is something we have seen students — and engineers with five years of experience — ship to production. Every right pattern is what your future code reviewer will expect. Treat this guide as a checklist; if your project violates one of these standards, fix it before submitting.

1 Async All the Way Down

The Problem

Your agent does almost nothing locally. It waits. It waits for the LLM to respond, for Postgres to return rows, for the weather API to answer, for the webhook to acknowledge. Every step in an agent loop is I/O — network calls and disk reads, not CPU work.

Python has one event loop per process. If a single function in your request path blocks that loop, every other in-flight request stops dead until it finishes. A 2-second LLM call that runs synchronously means your server can handle exactly one user every two seconds. That is not a server; that is a queue with extra steps.

The Antipattern

This is what blocking I/O in an async route looks like. It will appear to work. It will pass your manual tests. It will fall over the moment two users hit it at the same time.

✗ Looks fine. It is not.

```
@app.post("/plan-trip")
async def plan_trip(query: str):
    weather = requests.get(WEATHER_URL).json()       # blocks
    flights = requests.get(FLIGHTS_URL).json()       # blocks
    response = openai.chat.completions.create(...)   # blocks
    return {"plan": response.choices[0].message}
```

The route is declared async, so FastAPI runs it on the event loop. But every line inside it uses blocking libraries. The event loop sits idle waiting for each call to return, and no other request gets serviced in the meantime. You have written a synchronous server that lies about being async.

The Right Pattern

Use async libraries everywhere I/O happens. httpx for HTTP, asyncpg or SQLAlchemy 2.x async for the database, the async methods on every LLM SDK. Then await them in parallel where possible.

✓ Real async. Real concurrency.

```
import asyncio, httpx
from openai import AsyncOpenAI

client = AsyncOpenAI()

@app.post("/plan-trip")
async def plan_trip(query: str):
    async with httpx.AsyncClient() as http:
        weather, flights = await asyncio.gather(
            http.get(WEATHER_URL),
            http.get(FLIGHTS_URL),
        )
    response = await client.chat.completions.create(...)
    return {"plan": response.choices[0].message}
```

Two improvements just happened. The two API calls now run concurrently instead of sequentially — if each takes 500ms, you saved 500ms. And while any of them is in flight, the event loop is free to service other requests. You have a real async server.

Common Pitfalls

- Calling time.sleep() in async code. Use await asyncio.sleep() instead. time.sleep blocks the loop.
- Using the requests library anywhere in a request path. Replace with httpx. requests is blocking and there is no async version of it.
- Heavy CPU work in an async function (parsing huge files, running ML inference on big inputs). The event loop cannot help with CPU work. Push CPU-bound calls to asyncio.to_thread() or a worker queue.
- Forgetting that scikit-learn predictions are CPU-bound. For the trip classifier, model.predict() is fast enough to run inline, but if you ever ship a heavy model, wrap it in to_thread().

> *You'll feel this when: your demo works perfectly with one user, then the bootcamp TA opens it from two tabs and everything grinds to a halt.*

2 Dependency Injection

The Problem

Every route in your app needs the same things: a database session, an LLM client, the current logged-in user, the agent executor, the embeddings model. Without a system, you end up with copies of "create a database connection" and "validate the JWT" pasted into every endpoint. When you change one, you forget the others. When you write a test, you cannot replace any of them.

Dependency injection means: declare what your function needs, and let the framework hand it to you. FastAPI builds this in with Depends(). It is the cleanest DI system in any Python web framework. Use it.

The Antipattern

✗ Globals everywhere. No tests possible.

```
engine = create_async_engine(DATABASE_URL)        # global
llm = AsyncOpenAI(api_key=os.getenv("OPENAI_KEY"))  # global

@app.post("/plan-trip")
async def plan_trip(query: str, request: Request):
    token = request.headers.get("Authorization")
    user = decode_jwt(token)                          # auth in route
    async with AsyncSession(engine) as session:       # session in route
        result = await llm.chat.completions.create(...)
        ...
```

Three things are wrong here. The engine and LLM client are module-level globals — they are constructed at import time, not on app startup, and you cannot replace them in tests. Auth logic lives inside the route handler, so every endpoint will copy-paste it. Session lifecycle is hand-managed in every endpoint. Multiply this by 20 routes and you have a maintenance disaster.

The Right Pattern

✓ Dependencies declared. Composable. Testable.

```
# dependencies.py
async def get_session() -> AsyncSession:
    async with SessionLocal() as session:
        yield session

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    session: AsyncSession = Depends(get_session),
) -> User:
    return await load_user_from_token(token, session)

def get_llm() -> AsyncOpenAI:
    return app.state.llm  # populated in lifespan, see Chapter 3

# routes.py
@app.post("/plan-trip")
async def plan_trip(
    query: str,
    user: User = Depends(get_current_user),
    session: AsyncSession = Depends(get_session),
```

```
    llm: AsyncOpenAI = Depends(get_llm),
):
    ...
```

The route declares its needs and FastAPI fulfills them. Auth is a dependency that other dependencies can themselves depend on (get_current_user uses get_session). Sessions are scoped to the request via the yield in get_session — they open and close automatically. In tests you call app.dependency_overrides[get_llm] = lambda: FakeLLM() and the entire app uses the fake. No monkey-patching.

Why This Matters

Dependency injection is the difference between a codebase that grows gracefully and one that calcifies. Every senior engineer you will work with thinks in terms of dependencies. Routes that take their dependencies as parameters are inherently testable, swappable, and easy to read — the function signature tells you exactly what it touches.

> *You'll feel this when: you try to write your first real test and realize you can't replace the LLM client without editing source files.*

3 Singletons — Done Right

The Problem

Some objects in your app should exist exactly once per process. The database engine. The loaded ML model. The connection pool to your vector store. The LLM SDK client. The sentence-transformer embedding model. These are expensive to create and safe to share, and creating them per-request is anywhere from wasteful (a new HTTP connection pool each time) to actively broken (loading a 400MB model from disk on every request).

The Antipattern

✗ Loads the model every request. Painful.

```
@app.post("/classify")
async def classify(features: dict):
    model = joblib.load("model.pkl")           # 400ms per request
    embedder = SentenceTransformer("...")      # 800ms per request
    return {"label": model.predict([features])[0]}
```

Every request reloads the model from disk. The first user waits a second; the second user also waits a second; the GPU memory thrashes; your free-tier hosting bills explode. The model has not changed between requests. You should load it once.

The naive fix is also wrong:

✗ Globals at import time. Order-of-import bugs.

```
# main.py
model = joblib.load("model.pkl")  # runs on import

@app.post("/classify")
async def classify(features: dict):
    return {"label": model.predict([features])[0]}
```

Now the model loads on import. Tests slow down because importing your app loads the model. If model.pkl is missing, your app cannot even be imported — it crashes before FastAPI starts. Configuration cannot influence what gets loaded because globals run before settings are read.

The Right Pattern

Use FastAPI's lifespan handler. Build singletons on startup, attach them to app state, dispose on shutdown. Expose them through dependencies.

✓ Lifespan + DI. The production pattern.

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # startup
    app.state.model = joblib.load(settings.MODEL_PATH)
    app.state.embedder = SentenceTransformer(settings.EMBEDDING_MODEL)
    app.state.llm = AsyncOpenAI(api_key=settings.OPENAI_KEY)
    app.state.engine = create_async_engine(settings.DATABASE_URL)
    yield
    # shutdown
    await app.state.engine.dispose()
```

```
app = FastAPI(lifespan=lifespan)

# dependencies.py
def get_model(request: Request):
    return request.app.state.model

# routes.py
@app.post("/classify")
async def classify(features: dict, model = Depends(get_model)):
    return {"label": model.predict([features])[0]}
```

The model loads exactly once, when the app starts. Settings drive what gets loaded. Shutdown disposes resources cleanly. Tests can override the dependency to inject a fake model. The route signature still declares what it needs.

Rule of Thumb

- Per-process singletons (lifespan): database engine, ML models, embedding models, LLM clients, HTTP client pools, agent executor, vector store connection.
- Per-request (yield in a dependency): database session, transaction, request-scoped context like the current user.
- Per-call (no caching): anything you compute from input data.

> *You'll feel this when: your demo's startup time is fine, but every individual request takes 1.5 seconds for no apparent reason.*

4 Caching — Use lru_cache and TTL Caches Where They Pay Off

The Problem

Some operations are expensive and deterministic: parsing a config file, building a regex, loading a small lookup table. Some operations are expensive and stable for short windows: "weather in Beirut today," "current EUR-USD rate." Calling them on every request burns latency, money, and rate-limit budget.

Caching is the right answer for both, but the wrong cache is worse than no cache — it serves stale data and creates bugs that only appear hours after deployment.

Two Tools, Two Use Cases

functools.lru_cache — for deterministic, in-process functions

✓ Settings should load once.

```
from functools import lru_cache

@lru_cache(maxsize=1)
def get_settings() -> Settings:
    return Settings()

# Now any call to get_settings() returns the same instance.
# Use it in dependencies:
def settings_dep() -> Settings:
    return get_settings()
```

lru_cache stores the return value keyed by the function's arguments. With maxsize=1 and no arguments, it becomes a singleton-as-a-function. Use it for: settings loaders, model file path resolvers, anything pure that's expensive to compute.

Do not use lru_cache for: anything that takes mutable arguments (it can't hash a dict), anything that should expire, anything where the cache key is not the inputs (e.g., a function that depends on the current time).

TTL caches — for external responses that go stale

✓ Cache the weather for 10 minutes.

```
from cachetools import TTLCache
from asyncio import Lock

weather_cache = TTLCache(maxsize=500, ttl=600)  # 10 minutes
weather_lock = Lock()

async def get_weather(city: str) -> dict:
    if city in weather_cache:
        return weather_cache[city]
    async with weather_lock:
        # double-check inside the lock
        if city in weather_cache:
            return weather_cache[city]
        result = await fetch_weather_from_api(city)
        weather_cache[city] = result
        return result
```

The TTL prevents indefinite staleness. The lock prevents the thundering herd — if 100 users ask for Beirut weather at the same instant, only one call hits the API. Document your TTL choice in the README; "10 minutes" is a decision, not a default.

Caching Decision Tree

- Is the input set small and the function pure? Use lru_cache.
- Is the data fresh-enough-for-some-window? Use TTLCache. Pick the window deliberately.
- Does it need to survive process restarts or be shared across replicas? You need Redis. (Probably not for this project.)
- Is the cost of stale data high (auth tokens, prices, anything regulated)? Don't cache, or cache for seconds with explicit invalidation.

> *You'll feel this when: a friend tells you the trip plan they got is the same one their colleague got two hours ago, in a different city.*

5 Configuration — pydantic-settings, Not Magic Strings

The Problem

Configuration is where projects rot. It starts with one os.getenv("OPENAI_KEY") in main.py. Then a model name hardcoded in three places. Then a fallback URL that nobody remembers adding. Six months later, no one can answer "what does this app need to run" without grepping through five files and asking the original author.

The fix is centralization and validation: one Settings class, every value typed, missing required values fail at startup, and the rest of your code imports from the Settings object — never from os.environ directly.

The Antipattern

✗ Magic strings everywhere. Silent failures.

```
# scattered across 12 files:
api_key = os.getenv("OPENAI_KEY")
model = "gpt-4o-mini"
fallback_model = "gpt-3.5-turbo"
db_url = os.environ["DATABASE_URL"]
embed_model = "all-MiniLM-L6-v2"
weather_url = "https://api.open-meteo.com/v1/forecast"
```

If OPENAI_KEY is missing, os.getenv returns None and your app crashes on the third request with a confusing error. If model names change, you grep and pray you found them all. If you want to use a different model in dev vs. prod, you can't — they're hardcoded.

The Right Pattern

✓ Typed, validated, single source of truth.

```
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict
from functools import lru_cache

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="forbid",
    )

    # required
    openai_key: str = Field(..., min_length=1)
    database_url: str

    # with sensible defaults
    cheap_model: str = "gpt-4o-mini"
    strong_model: str = "gpt-4o"
    embedding_model: str = "all-MiniLM-L6-v2"
    weather_url: str = "https://api.open-meteo.com/v1/forecast"
    weather_cache_ttl: int = 600

@lru_cache(maxsize=1)
def get_settings() -> Settings:
```

```
    return Settings()
```

Required values use Field(...) and the app refuses to start without them. Defaults are explicit. The whole config is one importable object with autocomplete in your IDE. Tests construct Settings(openai_key="test", database_url="...") directly — no environment variables required. The .env file is read in dev; in prod you set real environment variables and the same code works.

Why extra='forbid' Matters

With extra="forbid", a typo in your .env (OPNAI_KEY instead of OPENAI_KEY) raises an error on startup. Without it, the typo silently sets nothing and your app starts up confused. Always forbid unknown keys. Always.

> *You'll feel this when: your app crashes in production at 3am because a config var has been silently None for two weeks.*

6 Type Hints, Pydantic, and the Boundary

The Problem

Data flows into your system from places you do not control: user HTTP requests, LLM tool-call arguments, webhook responses, database rows. Inside your system, you control the data. Outside, anything goes — the LLM might send a string where you expected a number, the user might send None, the database might return a row missing a column.

Without explicit boundaries between trusted and untrusted data, validation gets sprinkled defensively through every function ("just in case"), and the whole codebase becomes paranoid. With explicit boundaries — Pydantic models at every entry point — validation happens once, at the edge, and everything inside trusts its types.

The Antipattern

✗ Defensive checks everywhere. Trust nothing.

```
def classify_destination(profile):
    if not profile:
        return None
    if not isinstance(profile, dict):
        return None
    if "climate" not in profile:
        return None
    climate = profile.get("climate")
    if not isinstance(climate, str):
        return None
    cost = profile.get("cost_index", 0)
    if not isinstance(cost, (int, float)):
        cost = 0
    # ...real logic finally starts here
```

This function is 80% defense and 20% logic. Its return type is unknowable. Every caller has to handle None. Errors are swallowed silently. This is what happens when validation has no defined boundary — it gets pushed down into every function in fear.

The Right Pattern

✓ Validate at the edge. Trust your types.

```
from pydantic import BaseModel, Field
from typing import Literal

class DestinationProfile(BaseModel):
    climate: Literal["tropical", "temperate", "cold", "arid"]
    cost_index: float = Field(..., ge=0, le=100)
    top_activities: list[str] = Field(..., min_length=1, max_length=10)
    region: str

class ClassificationResult(BaseModel):
    label: Literal["adventure", "relaxation", "culture", "budget", "luxury",
"family"]
    confidence: float = Field(..., ge=0, le=1)

def classify_destination(profile: DestinationProfile) -> ClassificationResult:
    # profile is already valid. Just do the work.
```

```
    features = profile_to_features(profile)
    label = model.predict([features])[0]
    proba = model.predict_proba([features])[0].max()
    return ClassificationResult(label=label, confidence=float(proba))
```

The function is short and does one thing. Its inputs are validated by Pydantic before it runs. Its output is typed. Tests can construct DestinationProfile directly and any invalid construction fails immediately. The LLM, when calling this as a tool, gets a clear schema to fill in — Pydantic generates the JSON schema for you.

Where to Put Your Boundaries

- HTTP request bodies: Pydantic model on the FastAPI route.
- Tool inputs in the agent: Pydantic model on each tool function.
- LLM structured outputs: Pydantic model passed to the SDK or used to parse responses.
- Webhook payloads: Pydantic model for what you send.
- Database rows: SQLAlchemy models give you typed access; convert to Pydantic at the API boundary if you serialize.

> *You'll feel this when: a tool fails at 2am because the LLM sent {"cost": "twenty"} and your model expected a float, and the error message points to a numpy operation 12 frames down the stack.*

7 Errors, Retries, and Failure Isolation

The Problem

Every external call in your agent — LLM, tool API, webhook, database — will fail. Networks blip. APIs return 500s. Rate limits engage. Your job is not to prevent failure (you can't); it is to handle failure well enough that one transient outage does not cascade into a broken request.

There are three layers of failure handling, and you need all three.

Layer 1 — Timeouts

Every external call gets a timeout. Without one, a hung connection hangs your entire request indefinitely.

```
async with httpx.AsyncClient(timeout=10.0) as client:
    response = await client.get(WEATHER_URL)
```

Layer 2 — Retries with Backoff

Transient failures often go away on their own. Retry with exponential backoff before giving up.

✓ tenacity makes this clean.

```
from tenacity import (
    retry, stop_after_attempt, wait_exponential, retry_if_exception_type
)

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=1, max=10),
    retry=retry_if_exception_type((httpx.TimeoutException, httpx.NetworkError)),
    reraise=True,
)
async def fetch_weather(city: str) -> dict:
    async with httpx.AsyncClient(timeout=10.0) as client:
        r = await client.get(WEATHER_URL, params={"city": city})
        r.raise_for_status()
        return r.json()
```

Three attempts, exponential backoff between them, retrying only on transient errors (you do not retry a 400 Bad Request — that will fail the same way forever). After three attempts, the original exception is re-raised.

Layer 3 — Failure Isolation in the Agent Loop

This is the subtle one. When a tool fails inside an agent, you have a choice: raise an exception (which crashes the agent) or return a structured error to the LLM (which lets it reason about the failure and recover).

✗ Crashes the agent on any tool failure.

```
async def live_conditions(city: str) -> dict:
    return await fetch_weather(city)  # raises on retry exhaustion
```

✓ Lets the agent decide what to do.

```
class ToolError(BaseModel):
    error: str
    retryable: bool
```

```
async def live_conditions(city: str) -> dict | ToolError:
    try:
        return await fetch_weather(city)
    except httpx.HTTPStatusError as e:
        return ToolError(
            error=f"weather API returned {e.response.status_code}",
            retryable=False,
        )
    except (httpx.TimeoutException, httpx.NetworkError) as e:
        return ToolError(
            error=f"weather API unreachable: {e}",
            retryable=True,
        )
```

Now the LLM sees "weather API unreachable" in the tool result and can either skip the weather information, retry the tool, or tell the user it could not get fresh weather. The agent loop continues. The user gets a partial answer, not a 500.

What Not to Do

- Do not catch a broad Exception and swallow it. Catch specific types.
- Do not retry non-transient errors (4xx). They will fail the same way every time.
- Do not retry without a maximum attempt count. You will lock up your event loop.
- Do not let the webhook failing break the user-facing response. Webhook failure is logged; the API still returns the trip plan.

> *You'll feel this when: a TA grades your project and the live API happens to be down for 30 seconds. A robust system survives that. A fragile one fails the demo.*

8 Code Hygiene

Project Layout

A 600-line main.py is a sign that no one is in charge of structure. Real projects organize by concern.

```
app/
    main.py             # FastAPI app, lifespan, mount routers
    config.py           # Settings class
    dependencies.py     # shared Depends() functions
    routes/
        auth.py
        trips.py
        admin.py
    services/
        agent.py        # LangGraph agent assembly
        rag.py          # retrieval logic
        classifier.py   # ML inference wrapper
    tools/
        rag_search.py
        classify_destination.py
        live_conditions.py
    models/
        db.py           # SQLAlchemy models
        schemas.py      # Pydantic request/response schemas
    db/
        session.py
        migrations/     # Alembic
    tests/
```

The point is not these exact names. The point is that someone reading your repo for the first time can guess where the agent code lives in five seconds.

Logging — Never Print

print() is for scripts. Real apps use a logger. JSON-structured logs are searchable, filterable, and compatible with every observability tool you will ever use.

✓ Replace every print with a structured log.

```
import structlog

log = structlog.get_logger()

async def run_agent(query: str, user_id: int):
    log.info("agent.run.start", user_id=user_id, query_length=len(query))
    try:
        result = await agent.ainvoke({"input": query})
        log.info("agent.run.success", user_id=user_id, tools_used=result["tools"])
        return result
    except Exception as e:
        log.exception("agent.run.failure", user_id=user_id, error=str(e))
        raise
```

Each log line is a JSON object with named fields. Searching for everything that happened to user 42 is one query. Counting agent failures over time is one query. With print statements, you grep through plain text and pray.

Linters and Formatters

Set up ruff (or flake8 + black) and run them in pre-commit. Configure the rules once and stop arguing about whitespace forever. A clean diff is easier to review and easier to trust.

```
# pyproject.toml
[tool.ruff]
line-length = 100
target-version = "py312"
select = ["E", "F", "I", "B", "UP", "ASYNC", "S"]
ignore = ["E501"]

# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.5.0
    hooks:
      - id: ruff
      - id: ruff-format
```

README

Your README is the front door. It explains the architecture (with the diagram), how to run the project locally, what configuration is required, and where the interesting code lives. It is not a tutorial; it is a map. Engineers who join the project a year from now will read it once and either understand the system or close the tab. Write for that engineer.

> *You'll feel this when: you open your own project three months later and cannot remember which file the agent loop is in.*

9 Tests — At Least the Critical Path

The Problem

You will not get to 100% coverage and you should not try. What you must have is a small set of tests that runs in seconds, fails loudly when you break something important, and runs automatically on every commit. The goal is not test discipline for its own sake; it is the ability to refactor without fear and to know within seconds whether you broke something.

What to Test

Three categories cover most of what matters for this project.

Pydantic Schemas — Cheap and High-Value

If your tool input schemas are wrong, your whole agent is wrong. Test them with both valid and invalid data.

```
def test_destination_profile_rejects_invalid_climate():
    with pytest.raises(ValidationError):
        DestinationProfile(
            climate="freezing",  # not in allowed Literal
            cost_index=50,
            top_activities=["hiking"],
            region="EU",
        )

def test_destination_profile_accepts_valid_input():
    p = DestinationProfile(
        climate="temperate",
        cost_index=50,
        top_activities=["hiking", "museums"],
        region="EU",
    )
    assert p.climate == "temperate"
```

Tools — Mock the LLM, Test the Logic

Each tool should be testable without calling the real LLM or hitting the real API.

```
@pytest.fixture
def mock_weather_api(monkeypatch):
    async def fake_fetch(city):
        return {"city": city, "temp_c": 24, "conditions": "sunny"}
    monkeypatch.setattr("app.tools.live_conditions.fetch_weather", fake_fetch)

async def test_live_conditions_returns_structured_data(mock_weather_api):
    result = await live_conditions("Beirut")
    assert result["temp_c"] == 24
    assert result["conditions"] == "sunny"

async def test_live_conditions_handles_api_failure(monkeypatch):
    async def fake_failing(city):
        raise httpx.TimeoutException("timeout")
    monkeypatch.setattr("app.tools.live_conditions.fetch_weather", fake_failing)
    result = await live_conditions("Beirut")
    assert isinstance(result, ToolError)
```

```
    assert result.retryable is True
```

End-to-End — One Happy Path Through the Whole Agent

With all external calls mocked, run a full request through the agent and assert that the right tools were called and the response is well-formed. This is your safety net for any change to the agent assembly.

Run Them Automatically

A test that does not run is a test that does not exist. Add a GitHub Actions workflow that runs on every push and pull request. If your tests don't run automatically, they will rot within a week.

```
# .github/workflows/test.yml
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: pytest -q
```

> *You'll feel this when: you make a small refactor on day five, all your tests still pass, and you ship with confidence instead of dread.*

Bringing It All Together

These nine standards are not nine separate things. They reinforce each other. Async needs DI to manage shared resources cleanly. DI needs lifespan singletons to know what to inject. Singletons need a typed Settings to decide what to load. Errors need types to know what "a tool failure" looks like in code. Tests need DI to mock dependencies.

The reason production codebases feel different from tutorial code is not that they use fancier libraries. It is that every part of the codebase respects the same set of standards, and the parts compose because of it. Your Week 4 project is the first time most of you will hold all of this in your head at once. It will feel like a lot. By Week 6 it will feel like the only sensible way to write a service.

A Self-Review Checklist

Before you submit, walk this list. If you can answer yes to every line, you are shipping engineering work.

- Every route, tool, and external call is async. No requests, no time.sleep, no blocking I/O in the request path.
- Every dependency (DB session, LLM, model, current user) is declared with Depends(). No globals.
- Heavy resources (model, embedder, engine) load once in lifespan and dispose on shutdown.
- lru_cache on deterministic helpers. TTL cache on at least one external call where it makes sense.
- All config goes through a Settings class. No os.getenv outside of it. extra="forbid" is set.
- Every external boundary has a Pydantic model. Tools have typed inputs and outputs.
- Every external call has a timeout, retries with backoff, and structured error returns from tools.
- Code is split into modules by concern. Logging is structured. Linter and formatter run on every commit.
- Pydantic schemas, tool logic, and one end-to-end agent flow are tested. Tests run in CI.

> *Engineering is the discipline of writing code that other people can change without fear. — Hasan*